

A REAL BUILD LOG

Building Mawaqif

What actually happened when I built a real prayer app with AI — the bugs, the 3am fixes, and the exact prompts that got me unstuck

CURRENTLY GOING THROUGH TESTING

INSIDE

Contents

INTRODUCTION

About Mawaqif — and About This Book

PART ONE — HOW I ACTUALLY BUILT THIS

The AI Workflow: Who Does What

The Mawaqif Build Loop

PART TWO — THE PROBLEMS I ACTUALLY FACED

1 The Onboarding Trap (Three Paths, One Bug)

2 Diagnosed ≠ Fixed

3 The Railway + Stripe 503

4 The Silent Environment Variable Gap

5 Your Testing Tools Aren't Installed Until They Are

6 The Google Play Console Survival Guide

7 Apple Developer Enrollment with a Dead-End iPad

8 Paying a Stranger $5 to Break Your App

9 Deleting the Android SDK

10 The $20 Vault: Why I Left Vibe-Coding Platforms

What I'd Do Differently

CLOSING

Where Mawaqif Is Now — and Where You Come In

INTRODUCTION

About Mawaqif — and About This Book

Mawaqif is a prayer times and adhan app. It's currently going through testing — not finished, not perfect, still being patched based on real bugs from real testers. That matters, because this book isn't written from the finish line. It's written from inside the process, close enough to the mess that I can still remember exactly what broke and what I typed to fix it.

I've been vibe-coding for about a year. No CS degree, no bootcamp — just me, a stack of AI tools, and a lot of early mornings. I built Mawaqif waking up for Fajr, praying, then opening my laptop before the day got in the way. Somewhere in there, I also dealt with Railway deployment errors, a Stripe integration that quietly broke, a Play Console rejection, an Apple enrollment flow held hostage by my dad's ancient iPad, and a $5 Fiverr tester who found more bugs in one sitting than I found in weeks of testing it myself.

This book is not a coding course. I'm not going to explain what a variable is or what "the cloud" means. If you're vibe-coding your own app right now — using AI tools to build something real instead of just prototyping in your head — this book is for the exact moment you hit a wall I already hit.

Every chapter follows the same shape, because that's genuinely how the problem-solving happened in real time:

- **The Story** → what was actually happening
- **The Problem** → what specifically broke
- **The Fix** → what I actually did about it
- **The AI Workflow** → which tool I used, and why that one
- **Copy-Paste Prompt** → the exact wording you can reuse
- **Checklist** → the steps, in order
- **Verification** → how you know it actually worked
- **Lesson** → the one thing worth remembering

Nothing in here is invented to fill a page. If a chapter is short, it's because that's how much there actually was to say.

PART ONE

How I Actually Built This

The AI Workflow: Who Does What

Every tool in this workflow earned its spot by actually being useful for one specific kind of work — not because more tools looks more impressive.

CLAUDE

Primary development, and architecture/planning decisions. This is where new features and structural decisions actually get built and thought through.

DEEPSEEK

Debugging and bug hunting. When something's broken and I don't yet know why, this is the second opinion.

QWEN

A second coding and debugging perspective — useful specifically when the first tool's fix doesn't fully hold up.

KIMI

Code refinement and extra features — polishing something that already works rather than building it from zero.

VS CODE + CLAUDE CODE

Where it all actually gets integrated, edited, and tested on the real project files — the place local changes actually happen, separate from whichever chat tool did the thinking. Claude Code also does the shipping work: it builds the .aab (Android) and .ipa (iOS) from the React Native Expo project, and uses the GitHub CLI for the repo side.

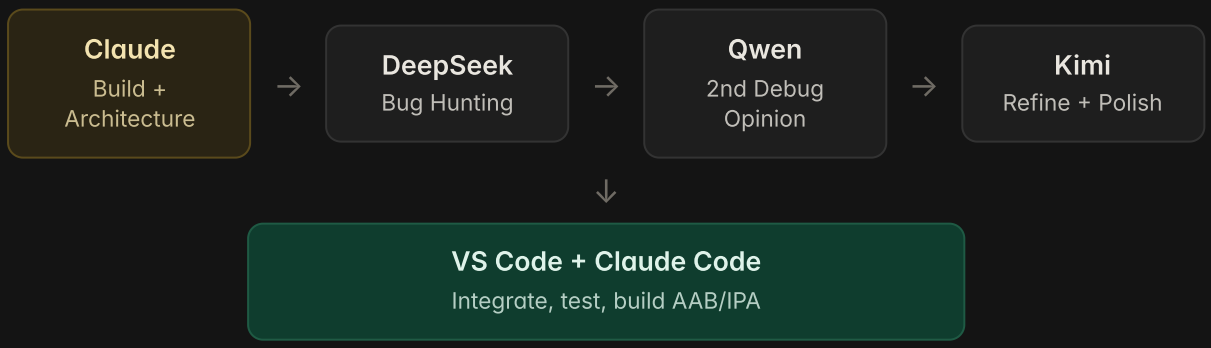


Fig. 1 — Feature work moves from Claude, through the debugging/refinement tools as needed, and lands in VS Code where it's actually integrated and tested.

LESSON

No single AI tool is "the" tool. Each one is useful for a specific kind of moment. The skill isn't picking the best one — it's knowing which moment you're in.

The Mawaqif Build Loop

This is the actual shape of the project. Not a straight line from idea to launch — a loop. Every chapter in Part Two is one lap of it.

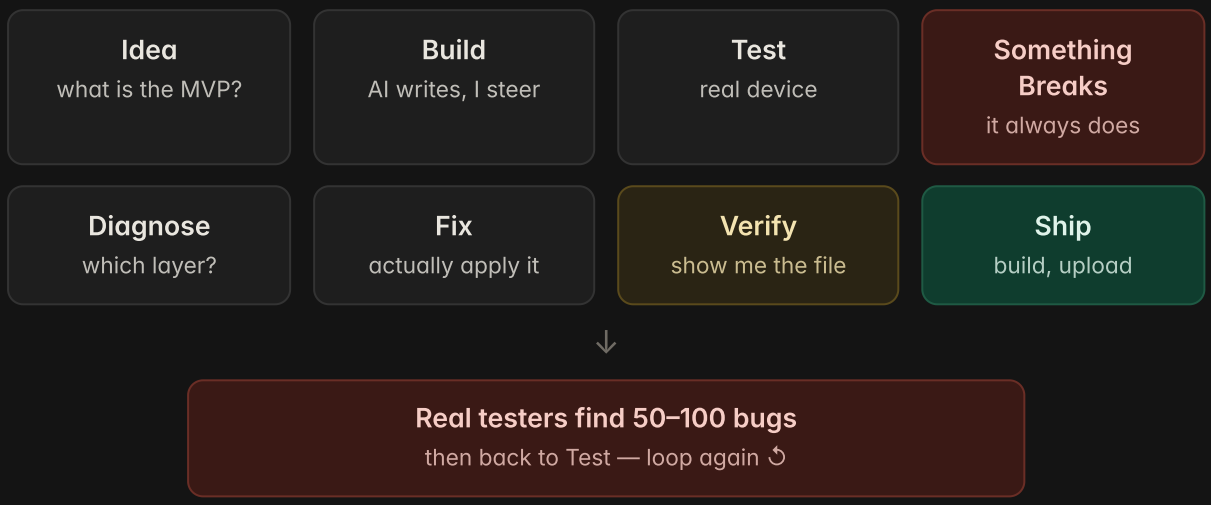


Fig. 5 — The loop Mawaqif went through, and is still going through while it's in testing.

LESSON

Shipping isn't the end of the loop. It's where real users start feeding it new bugs.

PART TWO

The Problems I Actually Faced

1 The Onboarding Trap — Three Paths, One Bug

THE STORY

> *A user could grant location, deny it, or manually search for a city during onboarding. One of those paths quietly broke everything downstream.*

THE PROBLEM

The app's redirect logic in `app/index.tsx` sent users back to onboarding whenever `state.location` was missing — not just when they weren't onboarded. That meant anyone who denied GPS permission (or never set a location) could get trapped, unable to reach Settings or the donate/support screen, even though neither screen actually required a location to work.

THE FIX

The redirect was changed to gate only on `!state.onboarded`, not on location. The onboarding flow was also updated so that denying GPS permission — or skipping location entirely — still sets `onboarded: true`, so the user isn't trapped on relaunch.

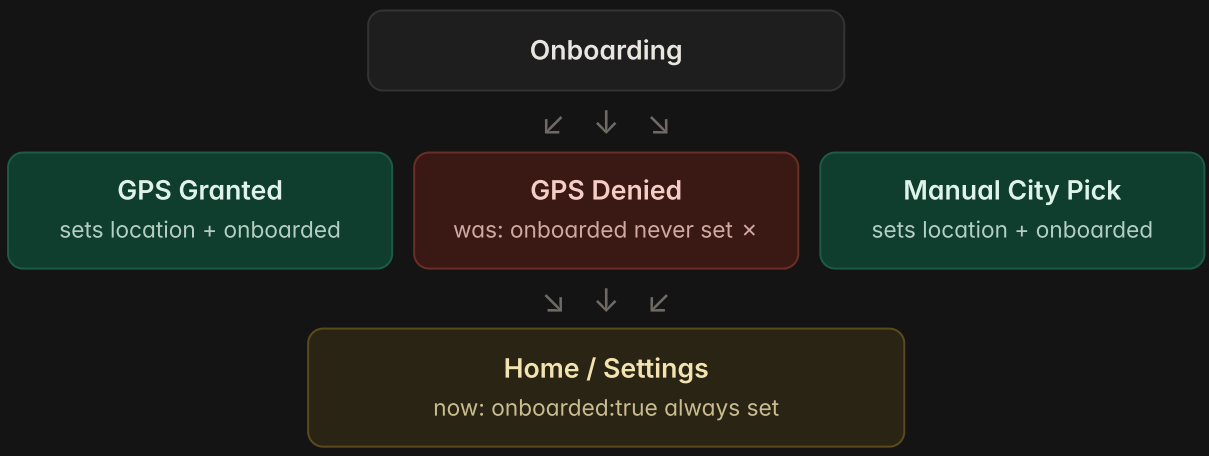


Fig. 2 — Three onboarding paths, one shared destination. Before the fix, the "GPS Denied" path never set onboarded: true, trapping the user.

THE AI WORKFLOW

Claude Code, working directly in the project files — this needed to see the actual redirect logic and onboarding flow, not just a description of the bug.

▶ COPY THIS PROMPT

```
Please apply this fix: in app/index.tsx, decouple the redirect logic from requiring a saved location. Currently it redirects to onboarding if (!state.onboarded || !state.location). Change this so it only redirects if !state.onboarded. Also update the onboarding flow so it sets onboarded: true even if the user denies GPS permission or skips setting a location, so they aren't stuck in an onboarding loop. Home and Settings already handle a null location gracefully. After making the change, show me a summary of exactly what changed before I request a rebuild.
```

CHECKLIST

- [ ] List every possible path through onboarding (granted, denied, manual, skipped) before assuming there's only one
- [ ] Check what each path sets — does it update both the "onboarded" flag and the location state?
- [ ] Confirm Settings and any other post-onboarding screen actually handle a missing location gracefully
- [ ] Ask the AI to trace each path individually rather than assuming a fix for one path covers all of them

VERIFICATION

Test all three paths on a real device: grant location, deny + skip, and manually pick a city. Confirm Settings and the donate/support screen are reachable after each one.

LESSON

Don't test only the happy path. A user can move through onboarding in more ways than you designed for, and every one of those ways needs to actually work.

2 Diagnosed ≠ Fixed

THE STORY

> *Claude walked through exactly what was wrong with the onboarding bug above — but the conversation moved on to a separate Stripe/Railway issue before the actual code change was ever applied.*

THE PROBLEM

A clear diagnosis feels like progress. It's easy to mentally file a bug as "handled" the moment an AI explains what's wrong — even though nothing in the codebase has actually changed yet.

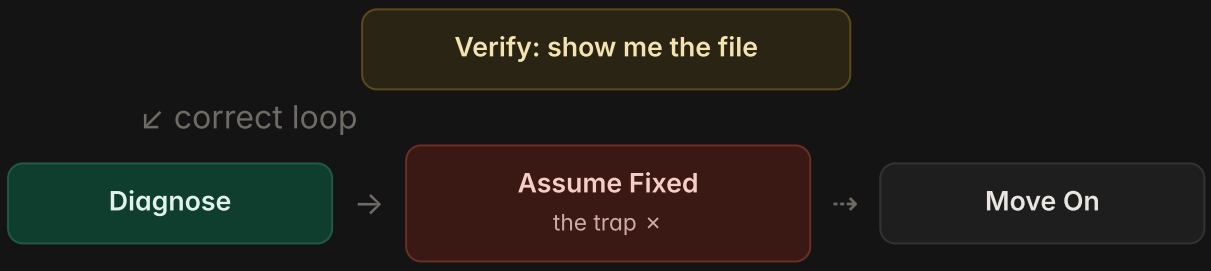


Fig. 3 — The trap is skipping straight from diagnosis to "move on." The correct loop inserts a verification step before anything is considered done.

THE FIX

Before triggering any rebuild, explicitly ask the AI to confirm the fix is in the files — not just repeat the plan back to you.

THE AI WORKFLOW

Whichever tool did the diagnosis — ask it to re-read the actual file and show you its current state.

▶ COPY THIS PROMPT

```
Before I rebuild, show me the exact current content of [file] — I want to
confirm the fix we discussed is actually applied in the code right now, not
just something we talked about.
```

CHECKLIST

- [ ] Never trigger a build or deploy directly after a diagnosis message
- [ ] Ask the AI to show the current file content, not restate the plan
- [ ] Compare it mentally against what the fix was supposed to change
- [ ] Only then run the build

VERIFICATION

The file the AI shows you actually contains the changed logic — not the original buggy version.

LESSON

A diagnosis is a sentence. A fix is a line of code that's actually been written. Don't confuse the two.

3 The Railway + Stripe 503

THE STORY

> *The donation feature in the app depends on Stripe. At one point, it started returning a 503 — "Stripe not configured" — even though the frontend code was fine.*

THE PROBLEM

The backend (hosted on Railway) was missing the Stripe API key. The frontend could look completely correct while the actual failure was sitting in the backend's environment configuration — a different layer of the stack entirely.

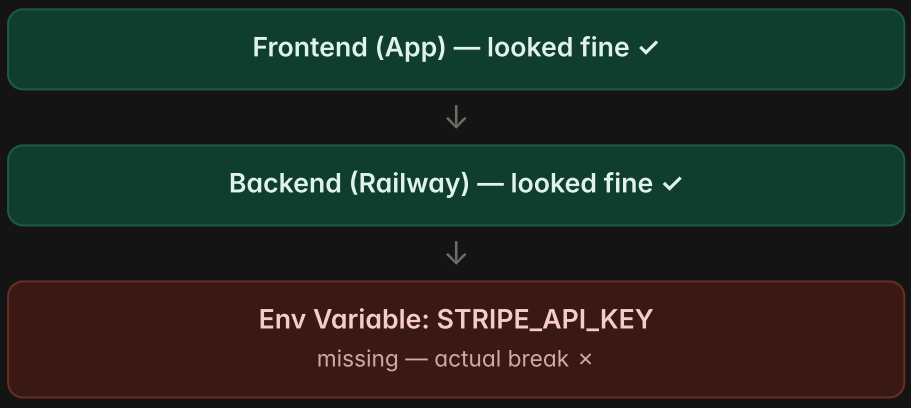


Fig. 4 — The failure wasn't in the layer that threw the visible error. It was one layer deeper, in configuration.

THE FIX

Set `STRIPE_API_KEY` directly as a Railway environment variable through Railway's dashboard. Because this is a configuration change, not a code change, no GitHub push was needed — Railway auto-redeployed as soon as the variable was saved.

THE AI WORKFLOW

Claude to diagnose which layer was actually failing (frontend vs. backend vs. environment config) before touching any code.

CHECKLIST

- [ ] When something fails in production, don't assume it's a code bug first

- [ ] Check the backend service's environment variables specifically
- [ ] Distinguish a config-only fix (no push needed) from a code fix (push required)
- [ ] Confirm the platform (Railway, in this case) actually auto-redeployed after the variable was saved

VERIFICATION

The donation flow completes without a 503, and Railway's deploy log shows a redeploy triggered by the environment variable change.

LESSON

When something fails in production, check the backend, the environment variables, and the deployment separately from the UI. The bug isn't always where the error message shows up.

4 The Silent Environment Variable Gap

THE STORY

> *The backend URL worked fine in production builds. It silently didn't exist in other build profiles.*

THE PROBLEM

`EXPO_PUBLIC_BACKEND_URL` was set under `build.production.env` in `eas.json` — but not under the preview or development profiles. Any build made from those profiles would bake in an undefined backend URL, with no obvious error until something tried to actually call the backend.

THE FIX

Add the same environment variable to the preview and development build profiles in `eas.json`, matching production.

▶ COPY THIS PROMPT

```
Add the same EXPO_PUBLIC_BACKEND_URL env var to the "preview" and
"development" build profiles in eas.json, not just "production" — so those
profiles don't silently build with an undefined backend URL.
```

CHECKLIST

- [ ] List every build profile your project actually uses (production, preview, development)
- [ ] Check that every environment variable the app needs exists in all of them, not just the one you test most
- [ ] Re-check this any time a new environment variable is added

VERIFICATION

Build from each profile and confirm the backend URL is defined and correct in each resulting build — not just production.

LESSON

Environment configuration is part of the application, not a small deployment detail you can set once and forget. It needs to be checked per build profile, every time.

5 Your Testing Tools Aren't Installed Until They Are

THE STORY

> *At one point, an attempt was made to run pytest to test part of the project.*

THE PROBLEM

Pytest wasn't actually installed in the environment. The test command didn't run the way it was expected to — meaning nothing was actually verified, even though it looked like a testing step had happened.

THE FIX

Treat "I ran the tests" as a claim to check, not a fact to trust. Look at the actual command output, not just the summary sentence describing it.

CHECKLIST

- [ ] When an AI says it tested something, scroll up and check the actual command output
- [ ] If a required tool isn't installed, that's a real step to handle — not something to quietly skip past
- [ ] Re-run the test yourself if you're not fully sure it happened

VERIFICATION

The actual test command output shows tests running and passing/failing — not an error about a missing tool, and not just a text summary claiming success.

LESSON

"AI said it tested it" is not the same as "it was tested." Read the output, not the summary.

6 The Google Play Console Survival Guide

THE STORY

> *Mawaqif's .aab was generated and submitted for review — expecting the hard part to be over.*

THE PROBLEM

Play Console flagged three issues: duplicate permission entries in the app config, a stray `SYSTEM_ALERT_WINDOW` permission that wasn't actually needed, and a missing justification for `SCHEDULE_EXACT_ALARM` — required for any app that fires alarms at exact times, like an adhan.

THE FIX

Strip permissions down to only what the app actually uses, and write a plain-language justification for any "sensitive" permission Google flags.

THE AI WORKFLOW

Claude or DeepSeek to audit the manifest/app config for permissions that aren't tied to any real feature; Claude to draft the Play Console justification text so it reads like an actual explanation, not boilerplate.

▶ COPY THIS PROMPT

```
Here's my app.json/AndroidManifest [paste it]. My app is a prayer time and adhan alarm app. Find every permission I'm requesting, tell me which ones I don't actually need for the features I have, and flag any duplicates.
```

CHECKLIST

- [ ] Paste your full manifest/permissions list to an AI and ask what each one actually does
- [ ] Remove anything not tied to a feature you actually built
- [ ] For `SCHEDULE_EXACT_ALARM` specifically: write 2–3 sentences explaining why exact timing matters (prayer times are fixed clock times, not "roughly around then")

- [ ] Check for duplicate entries — easy to introduce when merging AI-generated config changes
- [ ] Re-submit and check Play Console's Policy status tab, not just your email

VERIFICATION

Play Console's review status changes from "Needs justification" / "Rejected" to "In review" or "Live" — check the Policy tab directly.

LESSON

Every "sensitive" permission needs a plain-English reason a human reviewer can read in five seconds. AI-generated configs love adding permissions you never asked for — audit before you submit, not after rejection.

7 Apple Developer Enrollment with a Dead-End iPad

THE STORY

> *Enrolling in the Apple Developer Program meant using a family Apple ID tied to an old iPad — one too old to run Apple's current developer tools.*

THE PROBLEM

The iPad's iPadOS version was too old to run the current Apple Developer app or portal properly. But the enrollment flow still sometimes needs to send a 2FA verification code to a trusted device tied to that same Apple ID.

THE FIX

Enroll from a regular browser (Chrome/Edge) on a Windows PC at developer.apple.com — it's a standard web form, not an app, so it doesn't need the newer OS. The old iPad doesn't need to run anything demanding; it only needs to be able to receive a 2FA code, which works on far older OS versions than the developer tools themselves require. Going directly to `developer.apple.com/programs/enroll/` skips the general documentation pages and lands straight on the actual enrollment/payment flow.

CHECKLIST

- [ ] Use a regular browser on any working computer for enrollment — you don't need Apple's app or a newer device
- [ ] Go directly to the enrollment URL rather than browsing from the general developer homepage
- [ ] Keep the old device that's tied to the Apple ID nearby — you likely still need it for a 2FA code, even if it can't run anything else

VERIFICATION

You reach the actual account-type selection (Individual vs. Organization) and the $99 payment step — not just the general "learn about development" documentation pages.

LESSON

An old device doesn't have to be capable of much to still be useful — sometimes it just needs to receive one text or one push notification, and that's enough.

8 Paying a Stranger $5 to Break Your App

THE STORY

> *Once Mawaqif was in a working state, a tester was hired on Fiverr for $5 to actually use the app as a real stranger would.*

THE PROBLEM

That round of testing — plus a further additional testing pass — surfaced somewhere between 50 and 100 bugs. That's the real gap between "works when I use it" and "works when someone who's never seen it before uses it."

THE FIX

I used Emergent, a vibe-coding platform, for this stage. It cost about $20, and it turned out to be a trap. That story gets its own chapter (chapter 10).

LESSON

A cheap outside tester will find more real problems in one sitting than weeks of testing it yourself — because you already know how to use your own app "correctly." A stranger doesn't, and that's exactly the point.

This chapter is intentionally short — the exact triage process (how the 50–100 bugs were sorted and worked through) is still being written up.

9 Deleting the Android SDK

THE STORY

> *At one point I installed the Android SDK, thinking that was the "proper" way to build and test an Android app.*

THE PROBLEM

It's insanely heavy. The setup wants around 16 GB of RAM and about 60 GB of storage. For a solo builder on a normal machine, that's a huge cost for something you don't actually need.

THE FIX

I deleted it and moved to a React Native Expo project, worked on through GitHub or the GitHub CLI, in VS Code or Claude Code. Claude Code builds the .aab and .ipa from that Expo project, so nothing heavy has to live on my machine.

THE AI WORKFLOW

Claude Code, running inside the Expo project, handles the builds and the GitHub side. The other AIs never touch the build setup.

▶ COPY THIS PROMPT

```
I'm building a React Native Expo app on a normal laptop and I don't want to install Android Studio or the full Android SDK. Set up my project so I can build a production Android .aab and an iOS .ipa using Expo, and use the GitHub CLI for the repo. Tell me exactly what to install and what to skip.
```

CHECKLIST

- [ ] Start the project as React Native with Expo instead of a native Android project
- [ ] Skip Android Studio and the local SDK unless you have a specific reason you need them
- [ ] Set up GitHub or the GitHub CLI for the repo
- [ ] Use VS Code or Claude Code as your editor
- [ ] Let Claude Code run the build commands and produce the .aab and .ipa

VERIFICATION

You end up with an .aab ready to upload to Play Console, and you never had to install the SDK or 60 GB of tooling to get it.

LESSON

You don't need the heavy native setup. An Expo project, GitHub, and Claude Code can get you to a store-ready build on a normal machine.

10 The $20 Vault — Why I Left Vibe-Coding Platforms

THE STORY

> *I used a vibe-coding platform called Emergent. It cost about $20, and it felt like the easy way to get an app built.*

THE PROBLEM

The backend went into what I can only describe as an inaccessible vault. I couldn't get in and fix bugs manually. No manual edits, no real access, nothing. When something broke, the only option was to ask the platform to fix it, and if it couldn't, I was stuck.

THE FIX

Stop using platforms that hide your code. Use tools that work on real files you can open, edit, and keep: Claude Code, Kimi Code, Qwen Code, DeepSeek, or Z Code, inside VS Code with the project on GitHub.

THE AI WORKFLOW

Any of the coding tools above can build and debug on a project you fully own. Pick one to build with and a second one to check its work, the same way the workflow in Part One does it.

▶ COPY THIS PROMPT

```
Before I build my app on [platform name], tell me: can I download the full source code for both the frontend and the backend? Can I edit every file by hand? Can I move it to my own GitHub repo? What happens to my code if I stop paying? Answer each one plainly.
```

CHECKLIST

- [ ] Before you start on any platform, confirm you can download the frontend AND the backend
- [ ] Confirm you can edit every file by hand, not only through the platform's chat
- [ ] Put the project in your own GitHub repo as early as possible

- [ ] Prefer coding tools that work on real files you own (Claude Code, Kimi Code, Qwen Code, DeepSeek, Z Code)
- [ ] Ask what happens to your code if you stop paying or the platform changes

VERIFICATION

You can open every file of your app on your own machine, change a line by hand, and run or build it without the platform being involved.

LESSON

If you can't open your backend and fix a bug by hand, you don't really own the app. Use tools that give you the files.

LOOKING BACK

What I'd Do Differently

If I rebuilt Mawaqif from day one, these are the six habits I'd start with — every one of them comes from a chapter you just read.

1 **Map every path before testing**

List every way a user can move through a screen (granted, denied, skipped, manual) before assuming one test covers it.

2 **Never rebuild before verifying the fix is in the files**

Ask the AI to show the file. A diagnosis is not a fix.

3 **Check environment variables per build profile**

Production working doesn't mean preview and development do.

4 **Read the test output, not the summary**

"AI said it tested it" only counts if you saw the command actually run.

5 **Audit permissions before the first submission**

Every sensitive permission needs a plain-English reason. Do it before Play Console asks, not after.

6 **Don't build on a platform that locks your code**

If you can't open the backend and fix a bug by hand, you don't own the app. Use coding tools that work on real files.

CLOSING

Where Mawaqif Is Now — and Where You Come In

Mawaqif is still going through testing right now. It's not finished — bugs are still being found and patched, and the Apple release is still in progress. That's the honest state of it, and honestly, that's the point of this whole book: real software gets built in exactly this messy, iterative, "fix one thing and find two more" way, even when AI is doing a lot of the heavy lifting.

If you're building your own app right now and you're stuck somewhere — a deployment error, a bug you can't pin down, a store submission that got rejected, or just not knowing which AI tool to reach for — I'm offering a free session to help you work through it directly. Not a sales pitch, just the same kind of help this book is built from.

[Add your contact link / booking link / Discord invite here before publishing.]